

# STUDENT INFORMATION SYSTEM

*Project Documentation & Technical Report*

---

|                          |                                                    |
|--------------------------|----------------------------------------------------|
| <b>Prepared By:</b>      | Musaib Ahmed                                       |
| <b>Registration No:</b>  | 242201022                                          |
| <b>Academic Program:</b> | BS Computer Science                                |
| <b>Semester:</b>         | 4th Semester                                       |
| <b>Course Title:</b>     | Computer Organization and Assembly Language (COAL) |
| <b>Submitted To:</b>     | Sohail Ahmed                                       |

---

## Abstract

The Student Information System (SIS) is a high-performance, console-based utility developed in low-level x86 Assembly Language using the Netwide Assembler (NASM) framework and DOS operational interrupts. Designed to replace inefficient, manual record-keeping tracking sheets, this system implements a clean, interactive four-option user choice menu to streamline basic academic data management. The software enables administrative operators to seamlessly initialize new student profiles, display a comprehensive master registry of all saved records, search for specific student records using target identifiers, and execute a safe application termination sequence. This project serves as a practical application of foundational assembly programming concepts, including low-level data addressing, string arrays, iterative counter loops, conditional branching jumps, modular subroutines, and hardware-level interface integration via DOS interrupt execution.

## 1. Introduction

Educational institutions handle massive streams of student administrative and identity variables daily. Historically, relying on paper ledgers or bulky, manual filing systems introduced significant structural bottlenecks, heightened data entry errors, and reduced retrieval speeds.

To overcome these operational constraints, the Student Information System (SIS) introduces an execution-efficient, command-line architecture running within an emulated environment. The system organizes student data directly into low-level memory tables, granting operators instantaneous read and write access. Operating smoothly under an x86 microprocessor emulation layer via DOSBox, the software highlights the capabilities of low-level software engineering to deliver ultra-fast, responsive utilities without the overhead of massive modern software stacks.

## 2. Project Objectives

The core structural and educational objectives of this technical system development include:

- **Menu-Driven Integration:** Designing an interactive terminal console operating on a rigid four-choice execution matrix.
- **Low-Level Operations:** Gaining practical experience in register manipulation, stack modifications, and pointer calculations using NASM compiler syntax.
- **Structured Storage:** Using low-level memory models to read, write, and temporarily store alphanumeric data arrays.

- **Search Routines:** Developing string comparison algorithms in Assembly to verify and isolate specific records out of data blocks.
- **Report Formulation:** Building data display procedures that read raw data segment variables and render clean, highly readable outputs.

### 3. Problem Statement

Traditional record management systems in small academic settings are vulnerable to physical damage, data inconsistencies, and slow lookup speeds. Administrative bodies need a centralized, responsive interface capable of handling identity parameters instantly. The system must specifically resolve these issues by delivering a structured, low-overhead software utility configured to execute four essential system states:

- Instantly onboarding and registering individual student profiles.
- Displaying a complete, compiled registry of existing records inside memory arrays.
- Searching through active data segments using key indices to find unique records.
- Safely exiting the terminal workspace and restoring control to the parent OS shell.

## 4. Development Environment

- **Programming Language:** x86 Assembly Language (Intel Syntax)
- **Compiler Tool:** Netwide Assembler (NASM) v2.x
- **Emulation Subsystem:** DOSBox x86 Virtual Environment
- **Host OS Platform:** Microsoft Windows
- **Target Hardware Architecture:** Intel 8086 Microprocessor Instruction Set

## 5. System Features & Modular Architecture

The functional architecture of the application relies entirely on an explicit, user-driven console selector. The system isolates tasks into the following subroutines:

### Choice 1: Add Student Record

This module controls the onboarding process for new records, using input pointers to populate specific data matrices. The application prompts the operator for individual data variables, writing inputs into structured variables:

- **Student Identifier (ID):** A unique numerical alphanumeric tag.
- **Full Legal Name:** Text string array storing the student's name.
- **Age Entry:** A fixed-length numeric field.
- **Class / Grade Level:** An alphanumeric identifier mapping the current course placement.

### Choice 2: View All Records

This module scans the pre-allocated data segment arrays to pull and format active profiles. When selected, the software reads memory pointers sequentially, outputting a clean layout of all stored profiles—including IDs, Names, Ages, and Classes—giving operators a complete view of the current database snapshot.

### Choice 3: Search Student Profile

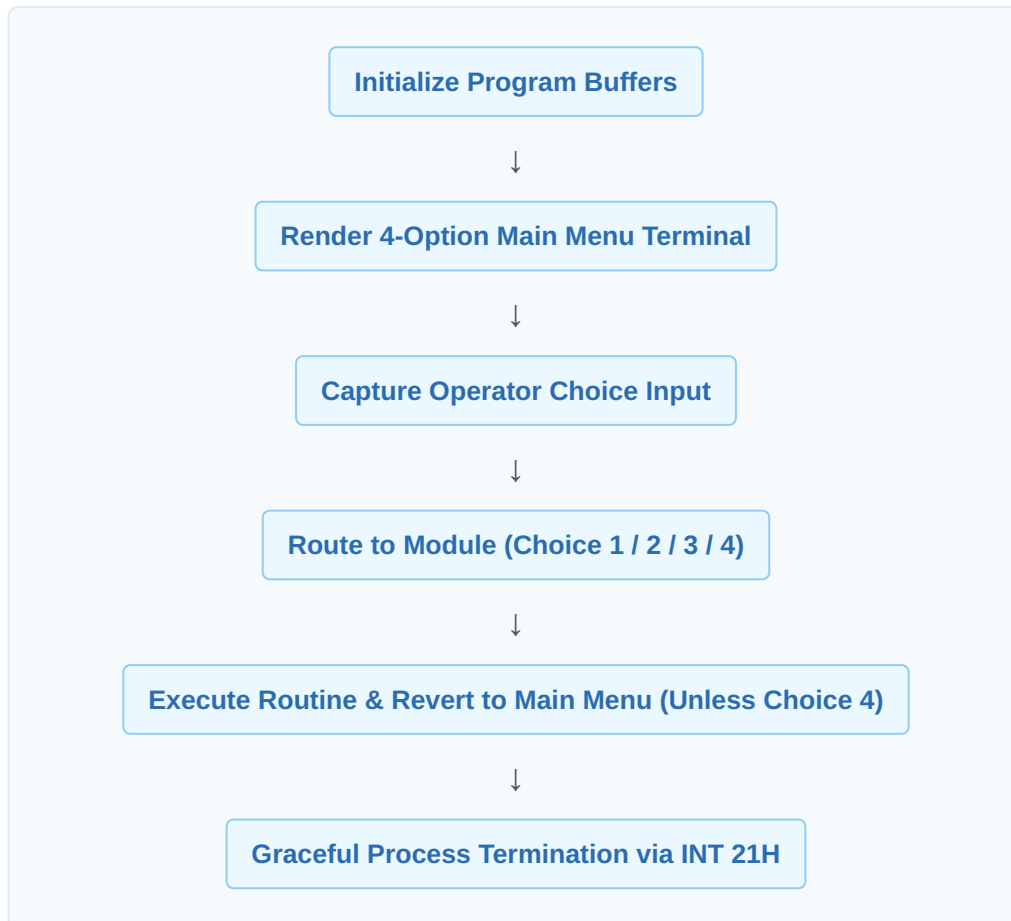
A targeted inquiry subroutine that instructs the operator to input a target query index (such as a specific Student ID). The application triggers a string comparison sequence, looping through stored record headers to find a match. If found, it outputs that student's complete profile details; if missing, it handles the error gracefully with an alert.

## **Choice 4: Exit Program**

This choice triggers a controlled shutdown sequence. It flushes standard input/output streams, unloads active memory pointers, and uses DOS interrupt function codes to terminate execution safely and return the operator to the standard command line prompt.

## 6. System Execution Flow

The internal state machine follows a strict loop structure to ensure execution remains stable and predictable until the exit command is called:



## 7. Technical Assembly Language Concepts Implemented

- **Data Segment Structuring:** Used to configure fixed-width storage spaces for strings, define interactive display messages, and maintain index matrices.
- **Procedural Subroutines (NEAR Calls):** Separates core tasks into reusable code blocks to keep code modular and readable. Key subroutines include PrintString, ReadString, and string comparison loops.
- **Loop Control and Counting:** Implemented via conditional flags and native assembly loop statements to handle index navigation, multiple character entries, and row parsing.
- **Conditional Branching Jumps:** Uses comparison operations (CMP) combined with conditional jump instructions (JE, JNE, JG) to handle menu selection logic and pinpoint exact index matches.

- **DOS Interrupt Application Services:** Directly relies on INT 21H vectors to communicate with host hardware, managing character reading, output generation, and system shutdown routines.

## 8. Core Algorithms

### 8.1 Student Registration Routine (Choice 1)

1. Display the alphanumeric request prompt for the Student ID.
2. Invoke ReadString to write input characters into the ID buffer.
3. Prompt the operator for the Student Name.
4. Capture character sequences to populate the Name data field.
5. Execute consecutive read loops to capture the student's Age and Class variables.
6. Advance tracking offsets to point to the next available slot in the data array.
7. Jump execution back to the Main Menu refresh loop.

### 8.2 Targeted Student Search Routine (Choice 3)

1. Prompt the operator to enter the target search ID.
2. Read the search string into a temporary comparison buffer.
3. Initialize the counter and point to the base address of the student record matrix.
4. Compare characters in the search string against the stored record index.
5. If characters match across the field width, branch to the match routine, display the full profile, and jump to step 8.
6. If they do not match, advance the base pointer by the record width offset.
7. Loop back to step 4 if more records exist; otherwise, display a "Record Not Found" alert.
8. Wait for a keystroke acknowledgment, clear working registers, and return to the Main Menu loop.

## 9. System Analysis & Performance Metrics

### 9.1 Advantages

- **Optimized Performance:** Executes instantly with direct register access, avoiding heavy software bloat.
- **Resource Efficiency:** Requires minimal memory overhead to manage internal data arrays.
- **Targeted Lookups:** The search routing enables immediate lookups by ID, eliminating the need to browse the entire list manually.
- **Streamlined UI:** The clear four-choice design prevents navigation errors and keeps tasks efficient.

### 9.2 Limitations

- **Volatile Memory Storage:** Data exists purely within active RAM; closing the application clears the records completely.
- **No Native Database Hooking:** The system lacks direct interfacing with relational or SQL database platforms.
- **Console-Only Interface:** Features a character-only terminal display with no graphical window elements.

### 9.3 Future Enhancements

- **Persistent File I/O:** Adding file-write interrupts (INT 21H file creation services) to save records into non-volatile text databases.
- **Dynamic Record Allocation:\*\* Moving beyond fixed matrix configurations to support variable-length dynamic lists.**
- **Enhanced Input Filtering:** Adding check filters to block invalid symbols or out-of-range numerical data.

## 10. Conclusion

The updated four-option Student Information System achieves its goals, showing how low-level Assembly code can solve core data management issues. By focusing on essential routines—Add Student, View All, and Search Student—the system provides a reliable tool without unnecessary performance overhead. Developing this system gave hands-on experience with register alignment, memory addressing offsets, conditional execution

**branches, and hardware interrupt calls, reinforcing key concepts in Computer Organization and Assembly Language.**